Tarea online PROG04

Título de la tarea: Tarea online PROG04

Unidad: PROG04

Ciclo formativo y módulo: Desarrollo de Aplicaciones Web. Programación

Curso académico: 2024/2025

¿Qué contenidos o resultados de aprendizaje trabajaremos?

Contenidos

PROG 04.- Cadenas de caracteres y arrays.

- Introducción a las estructuras de almacenamiento.
- Cadenas de caracteres.
  - Operaciones con cadenas de caracteres.
    - Operaciones básicas con cadenas de caracteres (I).
    - Operaciones básicas con cadenas de caracteres (II).
    - Operaciones avanzadas con cadenas de caracteres (I).
    - Operaciones avanzadas con cadenas de caracteres (II).
  - Conversiones entre cadenas y tipos numéricos (I).
    - Conversiones entre cadenas y tipos numéricos (II).
  - La clase StringTokenizer en Java.
  - Otros tipos de cadenas de caracteres en Java.
- Expresiones regulares.
  - Construcción de expresiones regulares.
    - Rangos.
    - Cuantificadores.
    - Agrupaciones: uso de paréntesis.
    - Alternativas: operador barra (|) o disyuntor.
    - Metacaracteres y escapado.
  - Expresiones regulares en Java (I): clases Pattern y Matcher.
    - El método matches de la clase String.
  - Expresiones regulares en Java (II): búsquedas.
  - Expresiones regulares en Java (III): agrupaciones.
- Creación de arrays.
  - Uso de arrays unidimensionales.
  - Inicialización.
  - Mostrar contenido de un array.
  - El método split de la clase String.
- Arrays multidimensionales.
  - Uso de arrays multidimensionales.
  - Inicialización de arrays multidimensionales.

Mostar el contenido de un array multidimensional.
Aplicaciones de los arrays multidimensionales.

Anexo I.- Formateado de cadenas en Java.

Anexo II.- Ejercicios resueltos.

Resultados de aprendizaje

- RA4. Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.
- RA6. Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos.

1.- Descripción de la tarea

Caso práctico

Ana está empezando a trabajar con clases y los conceptos de la programación orientada a objetos. Pero hay algunos de estos conceptos que aún no tiene del todo claros por no haber escrito todavía código Java que los utilice.

A **Juan** se le ha ocurrido mandarle a **Ana** hacer una aplicación para la gestión del sistema de matriculación instituto IES Al-Ándalus. El propósito de **Juan** es que **Ana** vaya adquiriendo soltura a la hora de implementar programas que aprovechen al máximo las ventajas de la programación orientada a objetos y por eso le han pedido que reflexione un poco sobre cómo organizaría las clases de una aplicación así. No hay nada como analizar una situación real para empezar a practicar de verdad con una herramienta. Eso es lo que va a intentar hacer **Ana**.

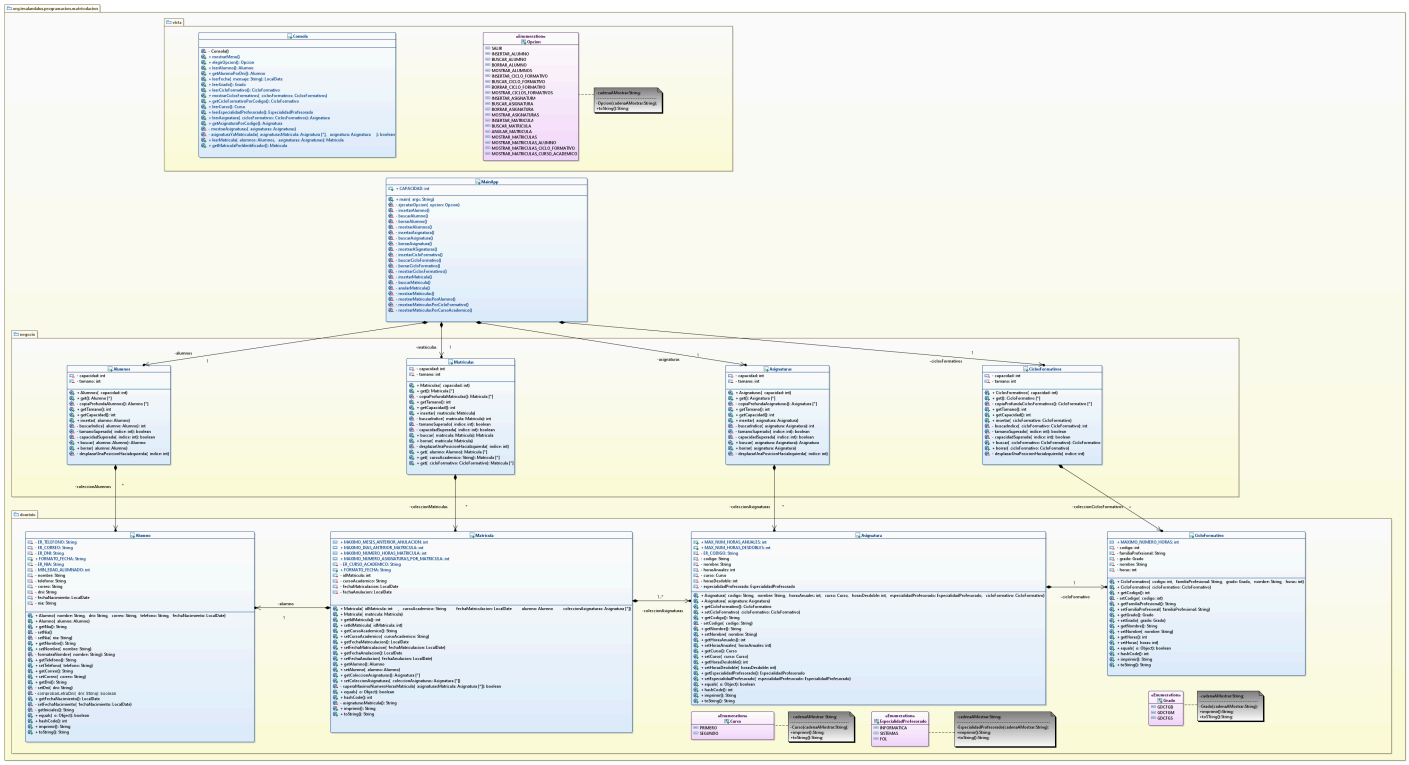
¿Qué te pedimos que hagas?

Tu tarea va a consistir en modelar la gestión del **sistema de matriculación del instituto IES Al-Ándalus**. Después de todas las especificaciones y requerimientos anotados, en este ***primer spring*** de la aplicación el equipo directivo del instituto quiere que gestionemos matriculación del alumnado del que se quiere guardar el **nombre**, un número de **teléfono**, el **email**, el **dni**, la **fecha de nacimiento** y el **Número de Identificación del Alumnado (NIA)**. Hay que tener presente que en el instituto se imparten **Ciclos Formativos** de diferentes familias profesionales y de diversos **Grados** (**Grado D Ciclo Formativo de Grado Básico, Grado D Ciclo formativo de Grado Medio y Grado D Ciclo Formativo de Grado Superior**). En referencia a las asignaturas, en este ***primer spring*** se ha decidido que se almacene la información referente al **código** que la identifica, su **nombre**, el **número de horas anuales** que se imparten de la asignatura en el ciclo formativo, el **número de horas de desdoble** (horas en las que hay dos profesores simultáneamente en clase), el **curso** en el que se imparte la asignatura (**Primero** o **Segundo**), la **especialidad** que debe tener el profesorado que imparte la asignatura y el **Ciclo Formativo** al que pertenece. Respecto a este último, el equipo directivo quiere almacenar un **código** que lo identifique, una cadena que proporcione el nombre de la **familia profesional** a la que pertenece el Ciclo, el **Grado** que le corresponde, su **nombre** y el total de **horas** del ciclo. Por último, en los ciclos formativos del **IES Al-Ándalus** solo pueden impartir docencia aquel profesorado que pertenezca a la especialidades de **Informática, Sistemas y Aplicaciones Informáticas y Formación y Orientación Laboral**.

Cuando a través de la aplicación se realice la matrícula de un alumno/a no solo se deberá almacenar la **lista de asignaturas** que constituyen la matrícula del alumno/a, también se deberá guardar el **identificador** de la matrícula, una cadena que represente el **curso académico** (debe tener formato dd-dd, por ejemplo, 23-24), la **fecha** en la que se realiza la matrícula, y en caso de que en un futuro se anule la matrícula, deberá quedar almacenada la **fecha de su anulación**.

He subido a GitHub un esqueleto de proyecto gradle que ya lleva incluidos todos los test necesarios que el programa debe pasar. La URL del repositorio es la siguiente: [repositorio GitHub con el esqueleto del proyecto](#).

Para ello te muestro un diagrama de clases para el mismo y poco a poco te iré explicando los diferentes pasos a realizar:



Primeros Pasos

Lo primero que debes hacer es realizar un **fork** del repositorio donde he colocado el esqueleto de este proyecto.
Clona tu repositorio remoto recién copiado en GitHub a un repositorio local que será donde irás realizando lo que a continuación se te pide. Modifica el archivo `README.md` para que incluya tu nombre en el apartado "Alumno". Realiza tu primer **commit**.

Alumno

Crea la clase `Alumno` en el paquete indicado en el diagrama:
Crea los diferentes atributos que se indican en el diagrama de clases con su visibilidad adecuada, teniendo en cuenta que la constante **FORMATO_FECHA** tiene como valor el formato de una fecha en España: **dd/MM/YYYY.**
Crea el método `formateaNombre`. Este método debe normalizar un nombre eliminando caracteres en blanco de sobra y poniendo en mayúsculas la primera letra de cada palabra y en minúsculas las demás.
Crea el método `comprobarLetraDni`. Este método hará uso de los grupos de las expresiones regulares (para ello has debido definir la expresión regular con grupos) para quedarse con el número por un lado y con la letra por otro. Para saber si la letra es válida puedes seguir las instrucciones del siguiente enlace: [Comprobar letra dni](#).
Crea los métodos de acceso y modificación de cada atributo con la visibilidad adecuada, teniendo en cuenta que un nombre estará compuesto de palabras separadas por un espacio y cada palabra comenzará con una mayúscula y continuará con minúsculas y que el **Nia** está formado por los cuatro primeros caracteres del nombre en minúsculas y los tres últimos dígitos del **DNI**. Además, ten presente que el **DNI**, el **teléfono** y el **correo** deben también tener un formato válido, debiendo comprobar que la letra del **DNI** sea correcta. Debes crear las constantes para las expresiones regulares que luego utilizarás en los métodos de

modificación. Los métodos de modificación lanzarán las excepciones adecuadas en caso de que el valor que se pretenda asignar al atributo no sea adecuado.
Crea el método **getIniciales**. Este método devolverá las iniciales del nombre.
Ten presente que para que se pueda registrar el alumnado en el sistema, este debe tener como mínimo 16 años **(MIN_EDAD_ALUMNADO).**
Crea el **constructor con parámetros** que hará uso de los métodos de modificación.
Crea el **constructor copia**.
Crea los métodos **equals** y **hashCode**, sabiendo que dos alumnos se considerarán iguales si su DNI es el mismo.
Crea el método **toString** que devuelva la cadena que esperan los tests.
Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

Curso

Crea el enumerado **Curso** teniendo en cuenta que sus posibles valores son **PRIMERO** y **SEGUNDO.**
Crea el atributo **cadenaAMostrar** (String).
Crea el constructor con la visibilidad adecuada, que almacenará en el atributo anterior el dato pasado por parámetro.
Crea el método **imprimir** indicado en el diagrama de clases que deberá devolver una cadena con el formato **dígito.-cadenaAMostrar**, para que se use en las opciones de menú. (0.-Primero y 1.-Segundo).
Crea el método **toString** que devuelva la cadena que esperan los tests.
Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

Grado

1. Crea el enumerado Grado teniendo en cuenta que sus posibles valores son los indicados con anterioridad.
2. Crea el atributo **cadenaAMostrar** (String).
3. Crea el constructor con la visibilidad adecuada, que almacenará en el atributo anterior el dato pasado por parámetro.
4. Crea el método **imprimir** indicado en el diagrama de clases que deberá devolver una cadena con el formato **dígito.-cadenaAMostrar**, para que se use en las opciones de menú.
5. Crea el método **toString** que devuelva la cadena que esperan los tests.
6. Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un commit.

CicloFormativo

Crea la clase **CicloFormativo** con los atributos y visibilidad adecuados.
Crea los métodos de acceso y modificación de cada atributo con la visibilidad adecuada, teniendo en cuenta que el identificador de una ciclo formativo será un número de cuatro dígitos. Además, debes crear las constantes indicadas en el diagrama de clases y que luego utilizarás en los métodos de modificación, sabiendo que el número máximo de horas de un ciclo formativo son 2000 horas. Los métodos de modificación lanzarán las excepciones adecuadas en caso de que el valor que se pretenda asignar al atributo no sea adecuado.
Crea los **constructores con parámetros** que harán uso de los métodos de modificación.
Crea el **constructor copia**.
Un ciclo formativo será igual a otro si su identificador es el mismo. Basándote en ello crea los métodos **equals** y **hashCode**.
Crea el método **toString** que devuelva la cadena que esperan los tests.
Crea el método **imprimir** que devuelva la cadena que esperan los tests.
Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

EspecialidadProfesorado

Crea el enumerado **EspecialidadProfesorado** teniendo en cuenta que cada instancia del enumerado puede ser el indicado en el diagrama de clases.
Crea el atributo con la visibilidad adecuada.

Crea el **constructor** con la visibilidad adecuada, que almacenará en el atributo anterior el dato pasado por parámetro.
Crea el método **imprimir** indicado en el diagrama de clases que deberá devolver una cadena con el formato **dígito.-cadenaAMostrar**, para que se use en las opciones de menú.
Crea el método **toString** que devuelva la cadena que esperan los tests.
Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

Asignatura

Crea la clase **Asignatura** con los atributos y visibilidad adecuados.
Crea los métodos de acceso y modificación de cada atributo con la visibilidad adecuada, teniendo en cuenta que el identificador de una ciclo formativo será un número de cuatro dígitos. Además, debes crear las constantes indicadas en el diagrama de clases y que luego utilizarás en los métodos de modificación, sabiendo que el número máximo de horas anuales de una asignatura son 300 horas y que el número máximo de horas de desdoble son 6. Los métodos de modificación lanzarán las excepciones adecuadas en caso de que el valor que se pretenda asignar al atributo no sea adecuado.
Crea los **constructores con parámetros** que harán uso de los métodos de modificación.
Crea el **constructor copia**.
Una asignatura será igual a otra si su identificador es el mismo. Basándote en ello crea los métodos **equals** y **hashCode.**
Crea el método **toString** que devuelva la cadena que esperan los tests.
Crea el método **imprimir** que devuelva la cadena que esperan los tests.
Comprueba que la **clase pasa los tes**t para la misma y cuando lo haga realiza un commit.

Matricula

Crea la clase **Matricula** con los atributos y visibilidad adecuados.
Crea los métodos de acceso y modificación de cada atributo, teniendo en cuenta que:

- El número máximo de horas de las asignaturas de una matrícula son 1000.
- El número máximo de asignaturas de una matrícula son 10.
- Como máximo, una matrícula se debe registrar con 15 días de retraso.
- La fecha de anulación de una matrícula no puede superar los 6 meses.
- El identificador de una matrícula debe ser un número positivo.
- El curso académico debe tener el formato **dd-dd**, por ejemplo, 23-24.

Crea el método **superaMaximoNumeroHorasMatricula** que deberá chequear si el total de horas de las asignaturas de la matrícula supera o no el máximo permitido.
Crea el **constructor con parámetros** que harán uso de los métodos de modificación.
Crea el **constructor copia**.
Una matrícula será igual a otra si tienen el mismo identificador. Basándote en ello crea los métodos **equals** y **hashCode.**
Crea el método **asignaturasMatricula** que devuelva la cadena que esperan los tests.
Crea el método **imprimir** que devuelva la cadena que esperan los tests.
Crea el método **toString** que devuelva la cadena que esperan los tests.
Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

Alumnos

Crea la clase **Alumnos**, en el paquete adecuado, con los atributos y métodos especificados en el diagrama y la visibilidad adecuada.

- Crea el **constructor con parámetros** que creará una lista de la capacidad indicada en el parámetro e inicializará los atributos de la clase a los valores correspondientes.
- El método **get** devolverá una copia profunda de la colección haciendo uso del método **copiaProfundaAlumnos.**
- Se permitirán **insertar** alumnos no nulos al final de la colección sin admitir repetidos.
- El método **buscar** devolverá un alumno si éste se encuentra en la colección y null en caso contrario.
- El método **borrar**, si el alumno se encuentra en la colección, lo borrará y desplazará los elementos hacia la izquierda para dejar el array compactado.

Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

CiclosFormativos

Crea la clase **CiclosFormativos**, en el paquete adecuado, con los atributos y métodos especificados en el diagrama y la visibilidad adecuada.

Crea el **constructor con parámetros** que creará una lista de la capacidad indicada en el parámetro e inicializará los atributos de la clase a los valores correspondientes.

Crea el método **get** que devolverá una copia profunda de la colección haciendo uso del método **copiaProfundaCiclosFormativos.**

Se permitirán **insertar** ciclos formativos no nulos al final de la colección sin admitir repetidos.

El método **buscar** devolverá un ciclo formativo si éste se encuentra en la colección y null en caso contrario.

El método **borrar**, si el ciclo formativo se encuentra en la colección, la borrará y desplazará los elementos hacia la izquierda para dejar el array compactado.

Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

Asignaturas

Crea la clase **Asignaturas**, en el paquete adecuado, con los atributos y métodos especificados en el diagrama y la visibilidad adecuada.

Crea el **constructor con parámetros** que creará una lista de la capacidad indicada en el parámetro e inicializará los atributos de la clase a los valores correspondientes.

Crea el método **get** que devolverá una copia profunda de la colección haciendo uso del método **copiaProfundaAsignaturas.**

Se permitirán **insertar** asignaturas no nulas al final de la colección sin admitir repetidos.

El método **buscar** devolverá una asignatura si ésta se encuentra en la colección y null en caso contrario.

El método **borrar,** si la asignatura se encuentra en la colección, la borrará y desplazará los elementos hacia la izquierda para dejar el array compactado.

Comprueba que la **clase pasa los test** para la misma y cuando lo haga realiza un **commit**.

Matriculas

Crea la clase **Matriculas**, en el paquete adecuado, con los atributos y métodos especificados en el diagrama y la visibilidad adecuada.

Crea el **constructor con parámetros** que creará una lista de la capacidad indicada en el parámetro e inicializará los atributos de la clase a los valores correspondientes.

Crea el método **get** que devolverá una copia profunda de la colección haciendo uso del método **copiaProfundaMatriculas.**

Se permitirán **insertar** matrículas no nulas al final de la colección sin admitir repetidos.

El método **buscar** devolverá una matrícula si ésta se encuentra en la colección y null en caso contrario.

El método **borrar**, si la matrícula se encuentra en la colección, la borrará y desplazará los elementos hacia la izquierda para dejar el array compactado.

El método **get** que está sobrecargado y devolverá una colección de las matrículas realizadas por el alumno pasado por parámetro o una colección de las matrículas realizadas para el curso académico indicado como parámetro o una colección de las matrículas realizadas para el ciclo formativo indicado como parámetro.

Asegurate de que se pasan todos los **tests asociados** a esta clase. Realiza el **commit** correspondiente.

Opción

Crea el enumerado **Opcion** en el paquete indicado en el diagrama cuyas instancias corresponderán con cada una de las opciones del menú de nuestra aplicación. Cada instancia aceptará en su constructor una cadena correspondiente al mensaje que se mostrará por pantalla para dicha opción. Debes crear los atributos y métodos que se especifican en el

diagrama, teniendo en cuenta que el método **toString** devolverá la cadena correspondiente a la concatenación del ordinal de la opción, la cadena " .- " y el mensaje a mostrar por la opción en cuestión.
Realiza un **commit** con la creación de este enumerado.

Consola

Crea la clase **Consola** en el paquete indicado en el diagrama.
Crea su **constructor** teniendo en cuenta que es una clase de utilidades y que no tiene sentido instanciar objetos de la misma.
Crea el método **mostrarMenu** que mostrará las diferentes opciones de nuestro programa de forma automática teniendo en cuenta cada uno de los objetos instancias del Enumerado **Opcion** (insertar, buscar, borrar, ...).
Crea el método **elegirOpcion** que pedirá que elijamos la opción y devolverá la instancia del enumerado **Opcion** correspondiente.
Crea el método **leerAlumno** que nos pedirá los datos correspondientes a un alumno y devolverá un objeto instancia de dicha clase en el caso que los datos introducidos sean correctos o propague la excepción correspondiente en caso contrario.
Crea el método **getAlumnoPorDni** que nos pedirá el dni de un alumno y devolverá un objeto instancia de dicha clase con el dni introducido y con el resto de datos ficticios cumpliendo las restricciones de creación de un alumno. En caso contrario, deberá propagar la excepción correspondiente.
Crea el método **leerFecha** que nos pedirá que introduzcamos una cadena correspondiente a una fecha en el formato adecuado y devolverá el objeto **LocalDate** correspondiente. Esto se repetirá mientras la fecha introducida no sea válida.
Crea el método **leerGrado** que mostrará la lista de grados existentes y devolverá un objeto instancia de dicha clase.
Crea el método **leerCicloFormativo** que nos pedirá los datos correspondientes a un ciclo formativo y devolverá un objeto instancia de dicha clase en el caso que los datos introducidos sean correctos o propague la excepción correspondiente en caso contrario.
Crea el método **mostrarCiclosFormativos** que mostrará la lista de ciclos formativos que en ese momento estén registrados en el sistema.
Crea el método **getCicloFormativoPorCodigo** que nos pedirá el código de un ciclo formativo y devolverá una instancia de dicha clase con el código introducido y con el resto de datos ficticios cumpliendo las restricciones de creación de un ciclo formativo. En caso contrario, deberá propagar la excepción correspondiente.
Crea el método **leerCurso** que mostrará la lista de cursos existentes y devolverá un objeto instancia de dicha clase.
Crea el método **leerEspecialidadProfesorado** que mostrará la lista de especialidades existentes y devolverá un objeto instancia de dicha clase.
Crea el método **leerAsignatura** que nos pedirá los datos correspondientes a una asignatura y devolverá un objeto instancia de dicha clase en el caso que los datos introducidos sean correctos o propague la excepción correspondiente en caso contrario.
Crea el método **getAsignaturaPorCodigo** que nos pedirá el código de una asignatura y devolverá una instancia de dicha clase con el código introducido y con el resto de datos ficticios cumpliendo las restricciones de creación de una asignatura. En caso contrario, deberá propagar la excepción correspondiente.
Crea el método **mostrarAsignaturas** que mostrará la lista de asignaturas que en ese momento estén registrados en el sistema.
Crea el método **asignaturaYaMatriculada** que indicará si en la lista de asignaturas pasada como parámetro, existe la asignatura también pasada como parámetro.
Crea el método **leerMatricula** que nos pedirá los datos correspondientes a una matrícula y devolverá un objeto instancia de dicha clase en el caso que los datos introducidos sean correctos o propague la excepción correspondiente en caso contrario. Hay que tener en cuenta, que en la misma matrícula no puede aparecer más de una vez la misma asignatura.
Crea el método **getMatriculaPorIdentificador** que nos pedirá el identificador de una matrícula y devolverá una instancia de dicha clase con el código introducido y con el resto de datos ficticios cumpliendo las restricciones de creación de una matrícula. En caso contrario, deberá propagar la excepción correspondiente.

Realiza el **commit** correspondiente.

MainApp

Crea la clase **MainApp** en el paquete indicado en el diagrama.

Crea los diferentes atributos que se indican en el diagrama de clases con su visibilidad adecuada.

Crea el método **ejecutarOpcion** que dada una opción, la ejecute llamando al método correspondiente de la clase.

Crea el método **insertarAlumno** que nos pedirá los datos de un alumno, haciendo uso de la clase **Consola**, y lo insertará en la colección correspondiente si es posible o informará del problema en caso contrario.

Crea el método **buscarAlumno** que nos pedirá **solo** el dni de un alumno haciendo uso de la clase **Consola**, y nos mostrárá a dicho alumno o informará de que no existe o informará del problema de acontecido.

Crea el método **borrarAlumno** que nos pedirá **solo** el dni de un alumno haciendo uso de la clase **Consola**, y lo borrará si es posible de la lista de alumnos o informará del problema en caso contrario.

Crea el método **mostrarAlumnos** que mostrará todos los alumnos almacenados, si es que hay, o si no, nos informará de que no hay alumnos.

Crea el método **insertarAsignatura** que nos pedirá los datos de una asignatura, haciendo uso de la clase **Consola**, y lo insertará en la colección correspondiente si es posible o informará del problema en caso contrario.

Crea el método **buscarAsignatura** que nos pedirá **solo** el código de una asignatura haciendo uso de la clase **Consola**, y nos mostrará dicha asignatura o informará de que no existe o informará del problema acontecido.

Crea el método **borrarAsginatura** que nos pedirá **solo** el código de una asignatura haciendo uso de la clase **Consola**, y lo borrará si es posible o informará del problema en caso contrario.

Crea el método **mostrarAsignaturas** que mostrará todos las asignaturas almacenadas, si es que hay, o si no, nos informará de que no hay asignaturas.

Crea el método **insertarCicloFormativo** que nos pedirá los datos de un ciclo formativo, haciendo uso de la clase **Consola**, y lo insertará en la colección correspondiente si es posible o informará del problema en caso contrario.

Crea el método **buscarCicloFormativo** que nos pedirá **solo** el código del ciclo formativo haciendo uso de la clase **Consola**, y nos mostrará dicho ciclo formativo o informará de que no existe o informará del problema acontecido.

Crea el método **borrarCicloFormativo** que nos pedirá **solo** el código de un ciclo formativo haciendo uso de la clase **Consola**, y lo borrará si es posible o informará del problema en caso contrario.

Crea el método **mostrarCiclosFormativos** que mostrará todos los ciclos formativos almacenados, si es que hay, o si no, nos informará de que no hay ciclos formativos.

Crea el método **insertarMatricula** que nos pedirá los datos de una matrícula haciendo uso de la clase **Consola**, y lo insertará en la colección correspondiente si es posible o informará del problema en caso contrario.

Crea el método **buscarMatricula** que nos pedirá solo el identificador de una matrícula haciendo uso de la clase Consola, y nos mostrará dicha matrícula o informará de que no existe o informará del problema acontecido.

Crea el método **anularMatrícula** que mostrará la lista de matrículas existentes, debiéndose elegir la que se desea anular y una vez elegida, deberá establecer como fecha de anulación de dicha matrícula, la introducida por el usuario. En el caso de que no pueda ser anulable deberá de informarse de dicha circunstancia.

Crea el método **mostrarMatriculas** que mostrará todas las matrículas almacenadas, si es que hay o si no, nos informará de que no hay matrículas.

Crea el método **mostrarMatriculasPorAlumno** que mostrará todas las matrículas que el alumno indicado tiene registradas en el sistema. Si no tuviese nos informará de que no hay matrículas. Para indicar el alumno del que se desea obtener sus matrículas se deberá indicar **solo** el dni del alumno. Se hará uso de los métodos correspondientes de la clase **Consola.**

Crea el método **mostrarMatriculasPorCicloFormativo** que mostrará todas las matrículas que el ciclo formativo indicado tiene registradas en el sistema. Si no tuviese nos informará de que no hay matrículas. Para indicar el ciclo formativo del que se desea obtener sus matrículas se deberá mostrar la lista de ciclos formativos existentes e indicar solo el código del ciclo. Se hará de los métodos correspondientes de la clase **Consola.**
Crea el método **mostrarMatriculasPorCursoAcademico** que mostrará todas las matrículas del curso académico indicado. Si no tuviese nos informará de que no hay matrículas.
Crea el método **main** que nos mostrará el menú de la aplicación, nos pedirá una opción y la ejecutará mientras no elijamos la opción salir. En caso de salir, la aplicación mostrará un mensaje de despedida.
Realiza un **commit** con la creación de esta clase.
Finalmente, realiza el **push** hacia tu repositorio remoto en GitHub.

Se valorará:

- La indentación debe ser correcta en cada uno de los apartados.
- El nombre de las variables debe ser adecuado.
- Toda la implementación realizada deberá ajustarse a lo indicado en este apartado y a lo mostrado en el diagrama de clase. En caso contrario, lo que se use que no esté indicado en el diagrama de clase no será considerado y por tanto, la parte correspondiente no será evaluada.
- Se debe utilizar la clase **Entrada** para realizar la entrada por teclado.
- El programa debe pasar todas las pruebas que van en el esqueleto del proyecto y toda entrada del programa será validada, para evitar que el programa termine abruptamente debido a una excepción.
- La corrección ortográfica tanto en los comentarios como en los mensajes que se muestren al usuario.
- **Para calificar cada uno de los criterios de evaluación asociados a la tarea será imprescindible que el código compile correctamente y se pueda ejecutar. En caso contrario, los criterios de evaluación serán calificados con un 0.**

2.- Información de interés

Recursos necesarios y recomendaciones

- Un IDE de desarrollo (NetBeans, Eclipse o IntelliJ). En dicho IDE deberás haber instalado el plugin que permita trabajar con proyectos gradle.
- La URL del repositorios es la siguiente: repositorio GitHub con el esqueleto del proyecto.

Indicaciones de entrega

Deberás crear un archivo de texto que incluya las explicaciones sobre los puntos que las merezcan según tu criterio. Es imprescindible que en este documento incluyas la URL al repositorio GitHub que has debido crear para realizar esta tarea.

Empaquetarás en un archivo comprimido el documento de texto anterior y el proyecto creado con el IDE que hayas utilizado.

El envío del archivo comprimido anterior lo realizarás a través de la plataforma. El archivo se nombrará siguiendo las siguientes pautas:

Apellido1_Apellido2_Nombre_PROG_Tarea04

3.- Evaluación de la tarea

Criterios de evaluación implicados

Criterios de Evaluación RA4

e.- Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.
f.- Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.

Criterios de Evaluación RA6

a.- Se han escrito programas que utilicen arrays.
g.- Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.

¿Cómo valoramos y puntuamos tu tarea?

Rúbrica de la tarea		
Criterios de Evaluación	Calificación	Retroalimentación
4e		
4f		
6a		
6g		